SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	**Assessment:**	Pseudocode Writing task
Weightage:	30%	**Total Marks:**	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Shen Ivin Clement	**Reg. No.:**	192371023
Section / Batch:	2023-2027	**Date:**	26-08-2026

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time embedded system continuously receives large blocks of data from a high-speed I/O device. Write pseudocode to design a DMA-based data transfer algorithm that transfers data directly from the I/O device to main memory with minimal CPU involvement. The algorithm should initialize the DMA controller, configure source and destination addresses, specify the transfer size, initiate the transfer, and handle completion and error interrupts.

2. An embedded system is connected to multiple I/O devices, each capable of generating interrupts with different priority levels. Develop pseudocode for an interrupt handling mechanism that identifies and prioritizes vectored interrupts, determines the interrupting device, and dispatches the appropriate interrupt service routine. The algorithm should ensure that high-priority interrupts are serviced efficiently while maintaining proper handling of lower-priority requests.

3. A computer system communicates with different I/O devices having varying latency, bandwidth, and data transfer requirements. Write pseudocode for a dynamic I/O selection mechanism that chooses between memory-mapped I/O and I/O-mapped I/O based on the performance requirements of each device. The algorithm should evaluate latency and bandwidth requirements and select the mapping technique that provides efficient communication.

4. A computer system contains multiple devices that simultaneously request access to high-speed communication interfaces such as PCIe, USB, and Thunderbolt. Design pseudocode for a bus arbitration algorithm that manages competing access requests and allocates the appropriate interface to each device. The algorithm should consider device priority, bandwidth requirements, interface availability, and fairness to ensure efficient utilization of communication resources.

5. A real-time embedded system communicates with both low-speed devices, such as simple sensors, and high-speed devices, such as data acquisition units and storage devices. Write pseudocode for a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupt-driven communication for high-speed devices. The algorithm should identify the device type, select the appropriate I/O technique, initiate data transfer, and handle completion and error conditions while minimizing CPU involvement. Code of Conduct

I _____ *(Name / Reg No) certify that this submission is my original work and*

that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	**Weightage(%)**	**Level 4 – Exemplary (5 Marks)**	**Level 3 – Proficient (4 Marks)**	**Level 2 – Developing (2–3 Marks)**	**Level 1 – Beginning (0–1 Mark)**
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Pseudocode Control Structures (20)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	

SIMATS ENGINEERING

Department of Computer Science and Engineering

Assessment Tool 2 — Pseudocode Writing Task

Model Answer Key • CO5 (BL4 — Precision)

Course Code	CSA12	**Course Title**	Computer Architecture
CO Assessed	CO5 — Precision (BL4)	**Weightage**	30% · 100 Marks

CO5 Statement. Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe and Thunderbolt.

Each answer states the problem's inputs/outputs/constraints, gives structured pseudocode (the main deliverable), and closes with notes on the control structures used and the cases handled — matching the rubric: ***Problem Understanding, Algorithm Design, Use of Control Structures, Pseudocode Structure, Completeness****.*

Problem understanding — inputs / outputs / constraints

- **Input:** high-speed I/O device, destination buffer in main memory, transfer size
- **Output:** data moved device→memory with minimal CPU involvement
- **Constraints:** initialise DMA, set src/dest/size, start, handle completion & error interrupts

Pseudocode — DMA_Transfer + completion/error ISR

```
FUNCTION DMA_Transfer(device, memBuffer, size):
// 1. Initialise the DMA controller channel
 DISABLE_interrupts(device)
 DMA_reset(DMA_CH) // put channel in known state

 // 2. Configure source, destination and transfer size
 DMA_CH.source <- device.dataRegister // peripheral source
 DMA_CH.destination <- ADDRESS_OF(memBuffer) // main-memory dest
 DMA_CH.count <- size // bytes to transfer
 DMA_CH.direction <- PERIPHERAL_TO_MEMORY
 DMA_CH.mode <- BURST // block move, low CPU

 // 3. Enable completion + error interrupts
 DMA_CH.IE_complete <- TRUE
 DMA_CH.IE_error <- TRUE
 REGISTER_ISR(DMA_CH.vector, DMA_ISR)

 // 4. Start transfer, then release the CPU to other work
 DMA_CH.enable <- TRUE
 device.start()
 RETURN // CPU not involved in copy
END FUNCTION

ISR DMA_ISR(): // runs on transfer end/error
 IF DMA_CH.status.error THEN
 LOG("DMA error", DMA_CH.id)
 DMA_CH.enable <- FALSE
 SIGNAL(transfer_failed) // let caller retry/abort
 ELSE IF DMA_CH.status.complete THEN
 DMA_CH.enable <- FALSE
 FLUSH_cache(memBuffer, size) // make data visible to CPU
 SIGNAL(transfer_done)
 END IF
 CLEAR_interrupt(DMA_CH)
END ISR
```

Design notes — control structures & completeness

Uses a setup function + a separate ISR (modular structure); IF/ELSE-IF branches cover both completion and error paths.

CPU only initialises and is notified once at the end — satisfying the minimal-involvement requirement.

Problem understanding — inputs / outputs / constraints

■ **Input:** several devices raising interrupts with different priority levels + vector addresses ■ **Output:** correct ISR dispatched, high priority first, low priority not lost

■ **Constraints:** identify device, prioritise, dispatch vectored ISR, allow nesting

Pseudocode — prioritised, nested, vectored dispatch

```
FUNCTION Handle_Interrupts():
 WHILE interrupt_pending() DO
 dev <- HIGHEST_PRIORITY_pending() // controller arbitrates
 IF dev.priority > current_priority THEN
 PUSH(current_context) // save state (nesting)
 saved_pri <- current_priority
 current_priority <- dev.priority

 vector <- dev.vectorAddress // vectored: direct ISR addr
 ENABLE_interrupts_above(dev.priority) // higher may preempt
 DISPATCH(vector) // execute device ISR
 DISABLE_interrupts()

 current_priority <- saved_pri
 POP(current_context) // restore state
 ELSE
 QUEUE(dev) // defer lower-priority req
 END IF
 ACKNOWLEDGE(dev)
 END WHILE
 SERVICE_queued_lower_priority() // handled when CPU free
END FUNCTION

ISR Generic_Device_ISR(dev):
 READ dev.data
 CLEAR dev.flag
 SCHEDULE_softirq(dev) // heavy work deferred
END ISR
```

Design notes — control structures & completeness

WHILE loop drains all pending interrupts; IF/ELSE separates preempt-now from defer-later; nesting via context push/pop.

Lower-priority requests are queued and serviced later, so no request is dropped — completeness across priority levels.

Problem understanding — inputs / outputs / constraints

- **Input:** device with latency requirement and bandwidth requirement
- **Output:** chosen mapping technique + a matching access routine
- **Constraints:** evaluate latency & bandwidth, pick the more efficient mapping per device

Pseudocode — Select_IO_Mapping + Access

```
FUNCTION Select_IO_Mapping(device):
 lat <- device.latency_req_ns // smaller = stricter
 bw <- device.bandwidth_req_MBps

 IF (lat <= LATENCY_THRESHOLD) OR (bw >= BANDWIDTH_THRESHOLD) THEN
 mapping <- MEMORY_MAPPED_IO // fast load/store, high BW
 ELSE IF device.simpleControlOnly THEN
 mapping <- IO_MAPPED_IO // few regs, low bandwidth
 ELSE
 mapping <- MEMORY_MAPPED_IO // default to faster path
 END IF

 device.mapping <- mapping
 CONFIGURE_device(device, mapping)
 RETURN mapping
END FUNCTION

FUNCTION Access(device, reg, value):
 IF device.mapping = MEMORY_MAPPED_IO THEN
 MEM_WRITE(device.base + reg, value) // ordinary store
 ELSE
 OUT(device.ioPort + reg, value) // dedicated I/O instr
 END IF
END FUNCTION
```

Design notes — control structures & completeness

Decision made with IF / ELSE-IF / ELSE on measured latency and bandwidth thresholds. A second function abstracts the actual access so callers are independent of the mapping chosen — clean, reusable structure.

Problem understanding — inputs / outputs / constraints

- **Input:** competing device requests (priority, bandwidth need, preferred interface); interfaces with capacity/availability
- **Output:** each request granted a suitable interface
- **Constraints:** honour priority, bandwidth, availability and fairness (no starvation)

Pseudocode — Arbitrate + BEST_INTERFACE

```
FUNCTION Arbitrate(requests, interfaces):
// priority first, then aging (waitTime) for fairness
SORT requests BY priority DESC, waitTime DESC

FOR EACH req IN requests DO
iface <- BEST_INTERFACE(req, interfaces)
IF iface != NONE AND iface.freeBW >= req.bandwidthNeed THEN
GRANT(req.device, iface)
iface.freeBW <- iface.freeBW - req.bandwidthNeed
req.waitTime <- 0
ELSE
req.waitTime <- req.waitTime + 1 // aging prevents starvation
REQUEUE(req)
END IF
END FOR
ROTATE_priority() // round-robin fairness
END FUNCTION

FUNCTION BEST_INTERFACE(req, interfaces):
IF interfaces[req.preferred].available THEN
RETURN interfaces[req.preferred]
FOR EACH i IN interfaces ORDERED_BY freeBW DESC DO
IF i.available AND i.freeBW >= req.bandwidthNeed THEN
RETURN i
END IF
END FOR
RETURN NONE
END FUNCTION
```

Design notes — control structures & completeness

FOR-EACH over requests with a nested helper FOR-EACH over interfaces; IF/ELSE grants or ages each request.

Aging (waitTime) + ROTATE_priority guarantee fairness so no device is starved — all four criteria (priority, bandwidth, availability, fairness) handled.

Problem understanding — inputs / outputs / constraints

- **Input:** mixed device list — low-speed sensors and high-speed acquisition/storage units
- **Output:** data collected using the technique best suited to each device
- **Constraints:** poll low-speed, DMA+interrupt for high-speed, handle completion/error, min CPU

Pseudocode — Hybrid_IO with polling and DMA/interrupt paths

```
FUNCTION Hybrid_IO(deviceList):
 FOR EACH dev IN deviceList DO
 IF dev.type = LOW_SPEED THEN
 Poll_Device(dev) // simple sensors -> polling
 ELSE // HIGH_SPEED -> DMA + interrupt
 DMA_Interrupt_Transfer(dev)
 END IF
 END FOR
END FUNCTION

FUNCTION Poll_Device(dev):
 IF dev.status.ready THEN
 data <- MEM_READ(dev.dataRegister) // memory-mapped read
 STORE(data)
 END IF
 RETURN // quick, called in main loop
END FUNCTION

FUNCTION DMA_Interrupt_Transfer(dev):
 CONFIGURE_DMA(dev.source, dev.buffer, dev.size, BURST)
 ENABLE_DMA_interrupts(complete, error)
 START_DMA()
 RETURN // CPU free until interrupt
END FUNCTION

ISR HS_Complete_ISR(dev):
 IF DMA.status.error THEN
 HANDLE_error(dev) // retry / log / abort
 ELSE
 MARK_ready(dev.buffer) // block ready for the app
 END IF
 CLEAR_interrupt(dev)
 STOP_DMA(dev)
END ISR
```

Design notes — control structures & completeness

FOR-EACH + IF/ELSE routes each device to the right technique; polling and DMA paths are separate modular functions plus an ISR.

Both completion and error are handled and the CPU stays free during high-speed transfers — a complete, efficient hybrid design.